

RAG Documentation Assistant — Project Documentation

Audience: Software engineers transitioning into AI Engineering **Stack:** Python · LangGraph · ChromaDB · OpenAI · SentenceTransformers · BM25

Table of Contents

1. [What Is RAG and Why Does It Matter](#)
 2. [Core Concepts](#)
 - [2.1 Chunking](#)
 - [2.2 Embeddings](#)
 - [2.3 Vector Databases and HNSW](#)
 - [2.4 BM25 Keyword Search](#)
 - [2.5 Hybrid Search and Reciprocal Rank Fusion](#)
 - [2.6 LangGraph](#)
 - [2.7 Prompt Engineering for RAG](#)
 3. [Application Flow](#)
 - [3.1 Phase 1 — Ingestion](#)
 - [3.2 Phase 2 — Retrieval and Generation](#)
 4. [Code Walkthrough](#)
 5. [Design Decisions and Trade-offs](#)
 6. [How to Extend This Project](#)
-

1. What Is RAG and Why Does It Matter

Large Language Models (LLMs) like GPT-4 are trained on a snapshot of the internet up to a certain date. They have two fundamental limitations:

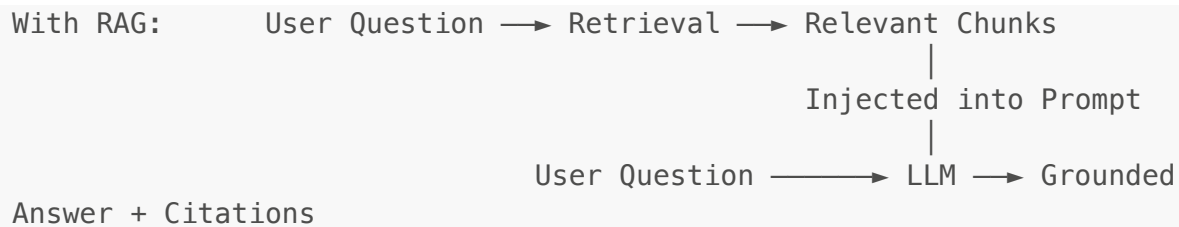
- **They do not know your private data** — your internal documentation, codebase, or proprietary knowledge base.
- **They hallucinate** — when they don't know an answer, they confidently fabricate one.

Retrieval-Augmented Generation (RAG) solves both problems. Instead of asking the LLM to recall an answer from training memory, you:

1. **Retrieve** the most relevant passages from your own documents.
2. **Inject** those passages directly into the prompt as grounding context.
3. **Generate** an answer that is constrained to only that context.

The LLM becomes a reasoning engine over your documents, not a knowledge store. If the answer is not in the retrieved context, a well-prompted RAG system will say so rather than hallucinate.

Without RAG: User Question —————> LLM —> Answer (may hallucinate)



2. Core Concepts

2.1 Chunking

An LLM cannot read your entire document library in one prompt — context windows are limited and large prompts are expensive. Chunking is the process of splitting documents into smaller, retrievable pieces.

The quality of your chunks directly determines the quality of your RAG system. Bad chunks = bad answers, regardless of how good your LLM is.

Character Splitting

The naive approach. A sliding window of fixed character count moves across the text, cutting wherever the count is reached.

```

Original text: "FastMCP registers a tool using the @mcp.tool() decorator.
                ```python
 @mcp.tool()
 def patient_risk_summary(patient_id: int) -> dict:
 ..."
```

Character chunk at 500 chars:

Chunk 1: "FastMCP registers a tool using the @mcp.tool()  
decorator.\n```python\n@mcp.tool()\ndef patient\_risk\_su"

Chunk 2: "mmmary(patient\_id: int) -> dict:\n ..." ← code block is split in half

#### Problems:

- Code blocks are cut mid-function.
- Tables are split across two chunks, breaking their structure.
- A single chunk often mixes content from two unrelated sections.
- The LLM receives incomplete, incoherent context.

#### Semantic Splitting

Splits at meaningful boundaries — in Markdown, this means `##` section headers. Each H2 section becomes one chunk, preserving all code blocks and tables within it.

```
Original text: "## 7. Building the MCP Server with FastMCP\n The MCP Python SDK provides...\n ```python\n @mcp.tool()\n def patient_risk_summary...\n ```"
```

Semantic chunk:

Chunk 1: Everything from "## 7. Building..." to the next "## 8." ← complete, intact

## Hybrid Chunking

A word-count gate applied on top of semantic splitting. If a section exceeds `SEMANTIC_MAX_WORDS` (400 words in this project), it is split further at `###` H3 sub-headers. This prevents any single chunk from becoming too large for the context window while still respecting meaning boundaries.

In this project, Section 6 of `MCP_Documentation.md` is the only section that triggers hybrid chunking — it contains four H3 tool descriptions that each become their own chunk.

---

## 2.2 Embeddings

An embedding is a fixed-size array of floating-point numbers that represents the **semantic meaning** of a piece of text. Two texts with similar meaning will have embeddings that are close together in vector space, even if they use different words.

```
"How do I register a tool?" → [0.21, -0.44, 0.87, ...] (384 numbers)
"What is the tool decorator?" → [0.19, -0.41, 0.85, ...] (close → similar
meaning)
"What is the weather today?" → [-0.72, 0.33, -0.12, ...] (far away →
different meaning)
```

This project uses `all-MiniLM-L6-v2` from the `sentence-transformers` library. It produces 384-dimensional vectors. It is small, fast, and runs locally — no API call required for embedding.

At ingest time, every chunk is embedded and stored alongside its text. At query time, the query is embedded using the same model, and the vector store finds chunks whose embeddings are closest to the query embedding.

---

## 2.3 Vector Databases and HNSW

A vector database is a data store optimized for **approximate nearest neighbor search** — finding the *N* vectors closest to a query vector out of potentially millions of stored vectors.

This project uses **ChromaDB** with a `PersistentClient`, meaning the data is written to disk (the `chroma_data/` folder) and survives restarts.

ChromaDB internally uses the **HNSW (Hierarchical Navigable Small World)** algorithm for indexing. HNSW builds a multi-layer graph where each node is connected to its nearest neighbors. To find the closest vector to a query:

1. Start at a random entry point in the top (sparse) layer.
2. Greedily move to the neighbor closest to the query vector.
3. Drop to the next layer and repeat — each layer is denser.
4. At the bottom layer, the current neighborhood is the approximate nearest neighbor set.

This runs in  $O(\log N)$  rather than  $O(N)$ , making it practical at scale. The trade-off is that it is **approximate** — it may occasionally miss the true nearest neighbor — but in RAG applications, this rarely matters because the second-closest chunk is usually just as good.

---

## 2.4 BM25 Keyword Search

BM25 (Best Match 25) is a classical information retrieval algorithm used in search engines. It scores documents based on **term frequency** and **inverse document frequency**:

- **Term Frequency (TF):** How often does the query word appear in this chunk?
- **Inverse Document Frequency (IDF):** How rare is this word across all chunks? Rare words (e.g., `get_patient_risk_summary`) score higher than common words (e.g., `the`, `is`).
- **Length Normalization:** Long chunks are penalized to prevent them from dominating just by containing more words.

The formula is:

$$\text{BM25}(\text{chunk}, \text{query}) = \sum_{\text{words}} \text{IDF}(\text{word}) \times \text{TF}(\text{word}, \text{chunk}) \times \frac{(k+1)}{\text{TF}(\text{word}, \text{chunk}) + k \times (1 - b + b \times \text{len}(\text{chunk})/\text{avglen})}$$

Where `k` and `b` are tuning constants (BM25Okapi defaults: `k=1.5`, `b=0.75`).

### BM25 is strong when:

- The query contains rare, specific tokens (`FastMCP`, `streamable_http_app`, `PRAGMA table_info`).
- The exact term appears only in one or two chunks.

### BM25 is weak when:

- The query is a natural language question with many common words.
- The query is conceptual ("`how does tool registration work?`") rather than lexical.

---

## 2.5 Hybrid Search and Reciprocal Rank Fusion (RRF)

Neither BM25 nor vector search is universally better. BM25 wins on exact tokens; vector search wins on semantic meaning. Hybrid search runs both and merges the results.

**Reciprocal Rank Fusion (RRF)** is the merging algorithm:

```
rrf_score(chunk) = 1 / (rank_in_bm25_list + 60)
 + 1 / (rank_in_vector_list + 60)
```

The constant **60** is a smoothing factor — it prevents top-ranked results from dominating too heavily and gives lower-ranked results a chance to surface when they appear in both lists.

**Example:**

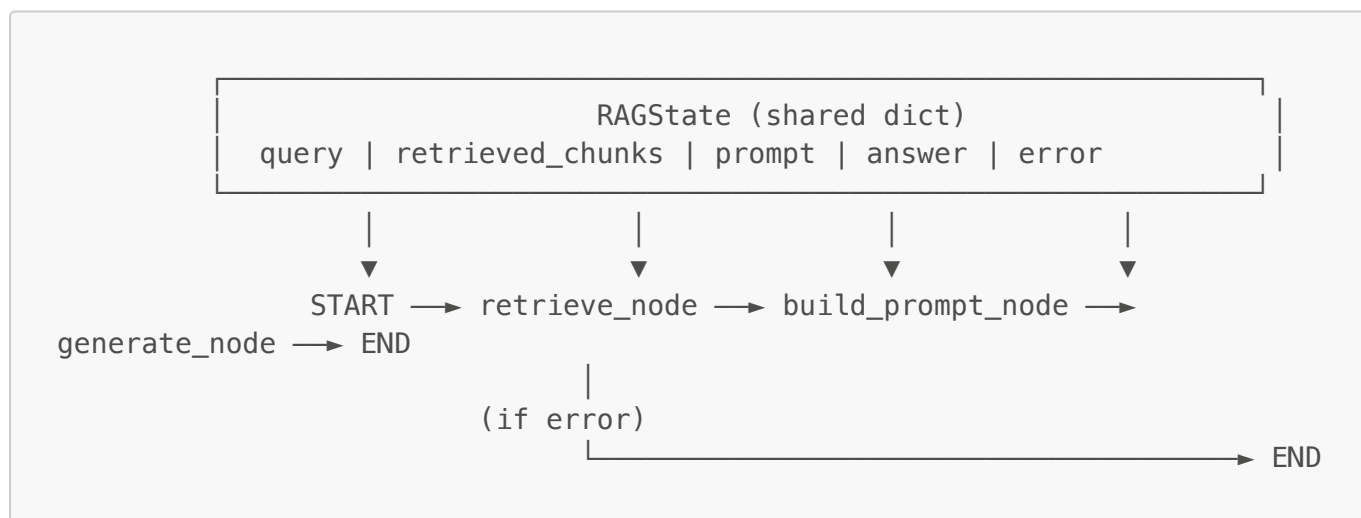
Chunk	BM25 Rank	Vector Rank	RRF Score
A	1	3	$1/61 + 1/63 = 0.0322$
B	5	1	$1/65 + 1/61 = 0.0318$
C	2	2	$1/62 + 1/62 = 0.0323 \leftarrow \text{wins}$

Chunk C wins because it appears in the top results of **both** systems. This is the key insight of RRF: **consensus across retrieval methods is more reliable than dominance in one.**

## 2.6 LangGraph

LangGraph is a library for building stateful, graph-based AI pipelines. It models your application as a **directed graph** where:

- **Nodes** are functions that perform one specific job (retrieve, build prompt, generate).
- **Edges** define the execution order between nodes.
- **State** is a shared dictionary that flows through every node, accumulating results.



Each node:

1. Receives the current state.
2. Does exactly one job.
3. Returns a dict with only the keys it updated.
4. LangGraph merges the returned dict back into the shared state.

Nodes **never call each other** — the graph wires them. This makes each node independently testable and swappable.

The conditional edge after `retrieve_node` checks `state["error"]`. If retrieval failed (e.g., the ChromaDB collection doesn't exist), the pipeline short-circuits directly to END rather than proceeding to prompt building with empty chunks.

---

## 2.7 Prompt Engineering for RAG

The system prompt is where hallucination is controlled. This project uses three explicit constraints:

1. Answer **ONLY** using the information in the provided document chunks.
2. If the answer is not present in the chunks, respond with:  
"I could not find an answer to this question in the provided documentation."
3. For every claim in your answer, cite the source using this format:  
[Source: <doc\_title>, Section: <section\_title>]

Constraint 1 prevents the LLM from using its training knowledge. Constraint 2 handles the "not in docs" case — the LLM refuses rather than invents. Constraint 3 makes every claim traceable back to a source chunk, enabling engineers to audit the answer.

The user message then injects the retrieved chunks in a labeled format:

```
--- Retrieved Documentation ---

[Chunk 1 - MCP_Documentation.md | 7. Building the MCP Server with FastMCP]
<full chunk text>

[Chunk 2 - DOCUMENTATION.md | Modern Approach: MCP_Maps_Agent]
<full chunk text>

...

Question: How does FastMCP register a tool?
```

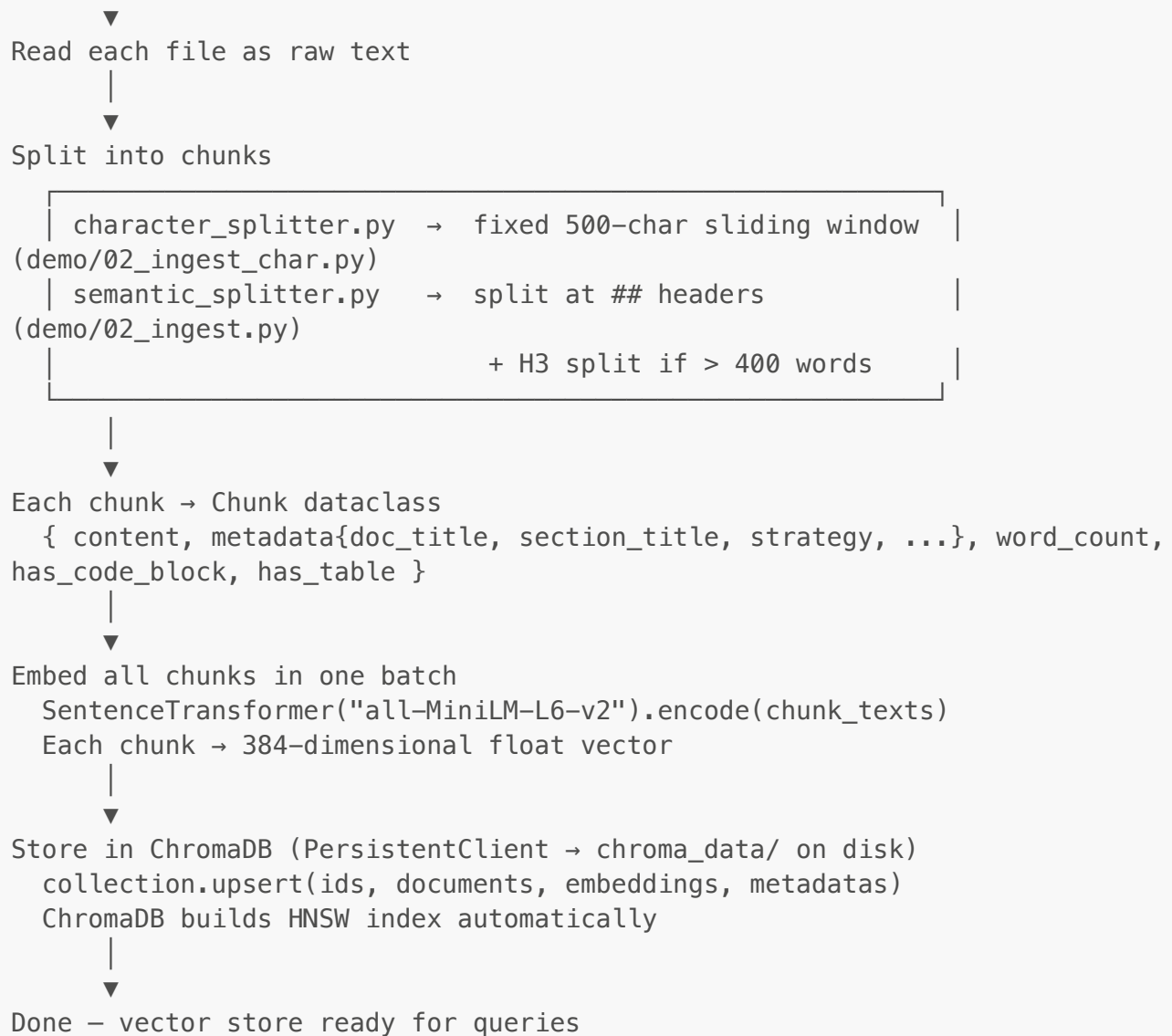
---

## 3. Application Flow

### 3.1 Phase 1 — Ingestion

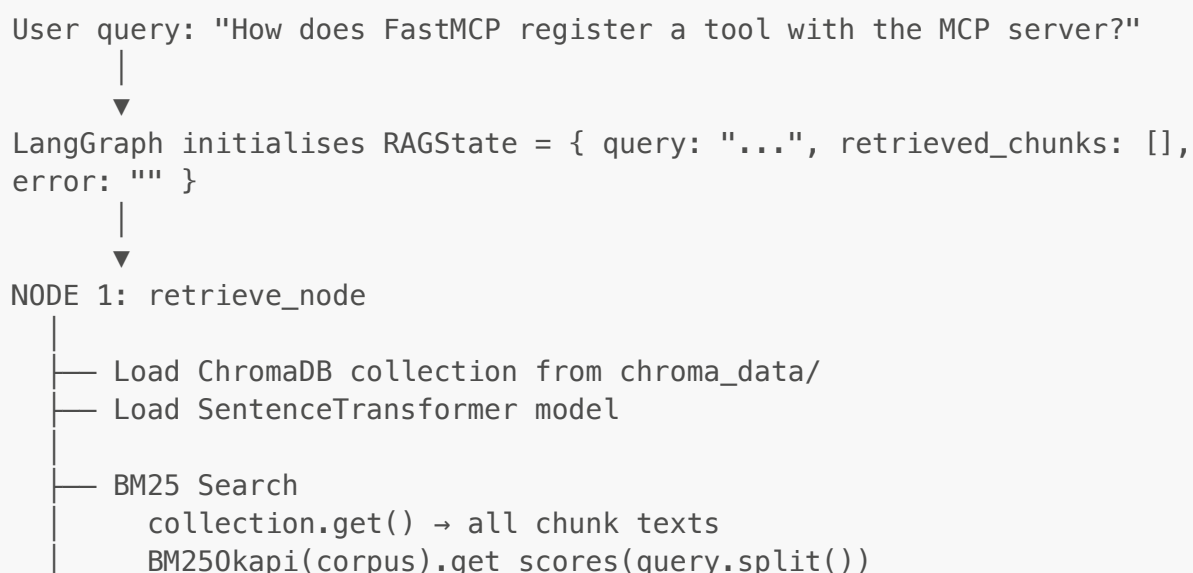
Ingestion is a one-time (or on-update) offline process. The output is a populated vector database on disk.

```
docs/*.md files
|
```



## 3.2 Phase 2 — Retrieval and Generation

This runs on every user query. The LangGraph pipeline executes the three nodes in sequence.



```

 Rank all chunks by BM25 score
 |
 |— Vector Search
 | model.encode(query) → 384-dim query vector
 | collection.query(query_embedding, n_results=10)
 | ChromaDB HNSW returns top-N by cosine distance
 |
 |— RRF Merge
 | rrf_score = 1/(bm25_rank+60) + 1/(vector_rank+60)
 | Sort all chunks by rrf_score descending
 | Return top 5 as List[RetrievedChunk]
 |
 | ▼ (if error → END)
 | ▼
 |
 | NODE 2: build_prompt_node
 | |
 | |— Assemble system prompt (grounding + citation instructions)
 | |— Build user message with labeled chunk texts + question
 | | Returns PromptMessages { system, user }
 | |
 | | ▼
 | |
 | | NODE 3: generate_node
 | | |
 | | |— client.chat.completions.create(model="gpt-4o-mini", messages=[system,
 | | | user])
 | | |— Extract citation strings with regex: [Source: ..., Section: ...]
 | | |— Return Answer { text, citations, query, chunks_used }
 | | |
 | | | ▼
 | | |
 | | | Print to terminal: answer text + citations + chunk count

```

## 4. Code Walkthrough

### config/settings.py

All constants live here. No functions, no classes — just values. Every other module imports from here instead of hardcoding values.

```

CHAR_CHUNK_SIZE = 500 # sliding window size for character splitting
CHAR_CHUNK_OVERLAP = 50 # overlap between consecutive character chunks
SEMANTIC_MAX_WORDS = 400 # H2 sections above this word count trigger H3
 splitting
TOP_K = 5 # number of chunks returned by retrieval
EMBED_MODEL = "all-MiniLM-L6-v2" # local sentence-transformer model

```

The overlap in character splitting is intentional — it ensures that a sentence split across two chunk boundaries appears in at least one chunk in its entirety. Semantic splitting does not need overlap because section boundaries are clean.

## chunking/character\_splitter.py

The `Chunk` dataclass is defined here and reused across the entire project — `semantic_splitter.py`, `embedder.py`, and `store.py` all import it.

```
@dataclass
class Chunk:
 content: str
 metadata: dict # doc_title, chunk_index, strategy, char_start,
 char_end
 word_count: int
 has_code_block: bool # True if "`" appears in content
 has_table: bool # True if "|---|" appears in content
```

`has_code_block` and `has_table` are detected by simple string checks. They are used by the comparison demo to count how many character-split chunks break a code block mid-way (odd number of ``` markers means the block was cut open).

```
def split_by_characters(text, doc_title, chunk_size, overlap):
 step = chunk_size - overlap # how far to advance the window each
 iteration
 start = 0
 while start < len(text):
 end = min(start + chunk_size, len(text))
 content = text[start:end]
 # ... build Chunk, append
 start += step # slide the window
```

## chunking/semantic\_splitter.py

Uses `re.split(r'\n(?:## )', text)` to split on H2 headers. The lookahead `(?:## )` keeps the `##` attached to the section text rather than discarding it.

The hybrid gate:

```
if word_count > max_words:
 # This section is too large – split further at H3 boundaries
 h3_parts = re.split(r'\n(?:###)', part)
 for h3_part in h3_parts:
 # each H3 becomes its own chunk
```

Each chunk's metadata records both `section_title` (the H2) and `subsection_title` (the H3, or empty string if no H3 split occurred). This is used by `prompt_builder.py` to label chunks clearly in the prompt.

---

## chunking/comparison.py

The key function here is the broken-code-block detection:

```
broken_code_blocks = sum(
 1 for c in chunks
 if c.has_code_block and c.content.count("`") % 2 != 0
)
```

A complete code block has an even number of `` markers (one to open, one to close). An odd count means the chunk contains an opened block with no closing marker — it was cut in half.

---

## vectorstore/embedder.py

```
def embed_chunks(chunks, model):
 texts = [chunk.content for chunk in chunks]
 embeddings = model.encode(texts, show_progress_bar=False)
 return [emb.tolist() for emb in embeddings]
```

`model.encode()` accepts a list of strings and returns a NumPy array of shape `(N, 384)`. The `.tolist()` call converts each NumPy row to a plain Python list — ChromaDB requires Python lists, not NumPy arrays.

All chunks are embedded in one batch call rather than one by one. Batch encoding is significantly faster because the model processes the texts in parallel on the GPU (or CPU SIMD).

---

## vectorstore/store.py

Three responsibilities:

**get\_or\_create\_collection** — used during ingestion. Safe to call multiple times; re-running ingestion upserts (updates or inserts) rather than duplicating.

**upsert\_chunks** — the document ID is constructed as `{doc_title}_{chunk_index}`. ChromaDB uses this ID for deduplication. If you re-ingest the same document, existing chunks are updated in place.

```
ids = [f"{chunk.metadata['doc_title']}_{chunk.metadata['chunk_index']}" for
 chunk in chunks]
```

**\_serialize\_metadata** — ChromaDB only accepts `str | int | float | bool` as metadata values. Python `None` would crash the upsert. This helper converts `None` to an empty string before storage.

**load\_collection** — used at query time. Raises a **RuntimeError** with a clear message if ingestion hasn't been run yet, rather than letting a cryptic ChromaDB exception bubble up.

---

## retrieval/semantic\_search.py

Defines the **RetrievedChunk** dataclass used everywhere downstream:

```
@dataclass
class RetrievedChunk:
 content: str
 metadata: dict
 score: float # distance from ChromaDB (lower = more similar)
 rank: int # 1-indexed position in the result list
```

The **search** function is a pure vector search — query embedding → ChromaDB HNSW lookup → ranked results. It is not used in the main pipeline (which uses hybrid search), but it is importable as an alternative. **hybrid\_search.py** imports **RetrievedChunk** from here to share the dataclass.

---

## retrieval/hybrid\_search.py

This is the most algorithmically complex file in the project.

### Step 1 — Fetch the entire corpus for BM25:

```
all_docs = collection.get(include=["documents", "metadatas"])
```

ChromaDB's HNSW index is designed for vector search. BM25 is a separate algorithm that needs the raw text of every document. There is no shortcut — you must fetch everything and build the BM25 index in memory at query time. For large corpora, this becomes a bottleneck (see trade-offs section).

### Step 2 — Build and score BM25:

```
tokenized_corpus = [doc.lower().split() for doc in documents]
bm25 = BM25Okapi(tokenized_corpus)
bm25_scores = bm25.get_scores(query.lower().split())
bm25_ranked = sorted(range(len(bm25_scores)), key=lambda i: bm25_scores[i],
reverse=True)
```

The corpus is tokenized by lowercasing and whitespace splitting. For production, you would use a proper tokenizer that handles punctuation (e.g., NLTK word\_tokenize).

### Step 3 — RRF merge:

```

rrf_scores: dict[str, float] = {}
for rank, corpus_idx in enumerate(bm25_ranked):
 doc_id = ids[corpus_idx]
 rrf_scores[doc_id] = rrf_scores.get(doc_id, 0.0) + 1.0 / (rank + 60)

for vid, _dist in zip(vector_ids, vector_distances):
 rank = vector_rank_map[vid]
 rrf_scores[vid] = rrf_scores.get(vid, 0.0) + 1.0 / (rank + 60)

```

Both loops accumulate into the same `rrf_scores` dict. A chunk that appears in both the BM25 list and the vector list will have contributions from both iterations — this is what causes consensus chunks to rise to the top.

---

## generation/prompt\_builder.py

Builds two strings: `system` and `user`. The system prompt is constant — it contains the grounding and citation instructions. The user message is dynamic — it changes with every query.

```

user_parts = ["--- Retrieved Documentation ---\n"]
for chunk in chunks:
 doc_title = chunk.metadata.get("doc_title", "Unknown")
 section_title = chunk.metadata.get("section_title", "Unknown")
 user_parts.append(f"[Chunk {chunk.rank} - {doc_title} | {section_title}]\n")
 user_parts.append(chunk.content)
 user_parts.append("\n")
user_parts.append("---")
user_parts.append(f"Question: {query}")

```

Each chunk is labeled with its source document and section. This labeling is what enables the LLM to produce accurate `[Source: ..., Section: ...]` citations — the information is right there in the prompt.

---

## generation/answer\_generator.py

```

response = client.chat.completions.create(
 model="gpt-4o-mini",
 max_tokens=1024,
 messages=[
 {"role": "system", "content": prompt.system},
 {"role": "user", "content": prompt.user},
],
)

```

```
text = response.choices[0].message.content or ""
citations = re.findall(r'\[Source: .+?, Section: .+?\]', text)
```

Citations are extracted with a regex after the fact. This is a simple approach — the LLM writes `[Source: X, Section: Y]` inline in its answer, and the regex collects all such patterns. If the LLM omits a citation or formats it differently, the regex silently misses it. A production system would use structured output (JSON mode) to enforce citation format.

---

## pipeline/state.py

```
class RAGState(TypedDict):
 query: str
 retrieved_chunks: List[RetrievedChunk]
 prompt: Optional[PromptMessages]
 answer: Optional[Answer]
 error: str
```

`TypedDict` gives the state a typed schema without the overhead of a full dataclass. Every node in the graph reads from and writes to this same dict. Using `TypedDict` means your IDE can autocomplete `state["query"]` and catch typos at development time.

`prompt` and `answer` are `Optional` because they are `None` in the initial state and only populated as nodes execute. `error` starts as an empty string — populated only if a node fails.

---

## pipeline/nodes.py

Each node follows the same three-line pattern:

```
def retrieve_node(state: RAGState) -> dict:
 try:
 # ... do the work
 return {"retrieved_chunks": chunks} # return only what changed
 except Exception as exc:
 return {"error": f"Retrieval failed: {exc}"}
```

The return value is a **partial state update** — LangGraph merges it into the full state. Nodes only return what they changed; they do not return the full state dict. This keeps each node's responsibility narrow and its return value easy to test.

---

## pipeline/graph.py

```
def build_rag_graph():
 graph = StateGraph(RAGState)
```

```

graph.add_node("retrieve_node", retrieve_node)
graph.add_node("build_prompt_node", build_prompt_node)
graph.add_node("generate_node", generate_node)

graph.add_edge(START, "retrieve_node")
graph.add_conditional_edges("retrieve_node", _route_after_retrieve)
graph.add_edge("build_prompt_node", "generate_node")
graph.add_edge("generate_node", END)

return graph.compile()

```

`add_conditional_edges` takes a routing function. The router inspects the state and returns the name of the next node (or `END`):

```

def _route_after_retrieve(state: RAGState) -> str:
 if state.get("error"):
 return END # short-circuit – skip prompt building and
 generation
 return "build_prompt_node"

```

`run_query` is the single public entry point for the entire pipeline. It builds the graph, invokes it, and returns the answer — callers do not need to know anything about LangGraph.

## 5. Design Decisions and Trade-offs

Decision	Why	Trade-off
<code>all-MiniLM-L6-v2</code> for embeddings	Fast, runs locally, no API cost	Lower accuracy than large models (e.g., <code>text-embedding-3-large</code> ). Swap in <code>embedder.py</code> with one line.
ChromaDB with HNSW	Zero config, persists to disk, good for prototyping	HNSW parameters ( <code>m</code> , <code>ef_construct</code> ) are automatic — not tunable. For production, use Qdrant or Weaviate with explicit index config.
BM25 built in memory at query time	Simple, no additional infrastructure	Fetches all documents from ChromaDB on every query. Slow for large corpora (10k+ chunks). For scale, maintain BM25 index as a separate file updated at ingest time.
RRF constant of 60	Standard default, robust across domains	Lower values (e.g., 10) make top-ranked results dominate more aggressively. Higher values (e.g., 100) smooth the merge more. Tune based on your retrieval quality metrics.
<code>gpt-4o-mini</code> for generation	Cheap, fast, good instruction following	Lower reasoning quality than <code>gpt-4o</code> . Swap model name in <code>answer_generator.py</code> .

Decision	Why	Trade-off
Citation extraction via regex	Simple, no structured output overhead	Brittle — if the LLM formats the citation differently, the regex misses it. Replace with OpenAI JSON mode for production.
LangGraph for the pipeline	Explicit state flow, easy to add nodes, built-in conditional routing	Overhead for a linear 3-node pipeline. The value increases as pipelines become more complex (loops, parallel branches, human-in-the-loop).

## 6. How to Extend This Project

**Swap the embedding model:** Change `EMBED_MODEL` in `config/settings.py`. Delete `chroma_data/` and re-run ingestion — embeddings from different models are not compatible.

**Add more documents:** Drop `.md` files into `docs/` and re-run `demo/02_ingest.py`. The ingestion script discovers all `.md` files automatically.

**Add a PDF loader:** Before chunking, convert PDF to Markdown using `pymupdf4llm` or `pdfplumber`. The rest of the pipeline is unchanged.

**Add a reranker:** Between retrieval and prompt building, add a fourth LangGraph node that passes the top-K chunks through a cross-encoder reranker (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`). Cross-encoders score query-chunk pairs jointly and are more accurate than bi-encoder similarity.

**Persist the BM25 index:** At ingest time, serialize the BM25 index to disk with `pickle`. At query time, load it instead of rebuilding from ChromaDB. This eliminates the full-corpus fetch on every query.

**Replace ChromaDB with Qdrant:** Only `vectorstore/store.py` changes. The three functions (`get_or_create_collection`, `upsert_chunks`, `load_collection`) are reimplemented using the Qdrant client. No other file changes.

**Add evaluation:** Use `RAGAS` (Retrieval-Augmented Generation Assessment) to measure faithfulness (does the answer match the chunks?), answer relevancy (does the answer address the question?), and context precision (were the right chunks retrieved?).